Back Propagation Feed Forward Network Practical Guide

Aim

To implement a Feed Forward Neural Network using the Back Propagation algorithm in Python.

Objectives

• To understand the Feed Forward Neural Network architecture.
• To understand the Back Propagation learning algorithm.
• To train a neural network using input and output datasets.
• To reduce prediction error by updating weights.
• To visualize learning using an error graph.

Theory

A Feed Forward Neural Network is one of the most basic forms of Artificial Neural Networks. Information moves in only one direction: Input Layer → Hidden Layer → Output Layer. Back Propagation is a supervised learning algorithm used for training neural networks. It calculates the error in prediction and updates the weights to minimize the error. In this practical, the XOR dataset is used to demonstrate learning using a hidden layer. XOR is a non-linear problem that cannot be solved using a single-layer perceptron.

Important Formulas

1. Sigmoid Function:
$\sigma(x) = 1 / (1 + e^{-x})$

2. Error Formula:
Error = Actual Output – Predicted Output

3. Sigmoid Derivative:
$\sigma'(x) = x(1 – x)$

Algorithm

1. Initialize random weights.
2. Perform forward propagation.
3. Calculate error.
4. Perform back propagation.
5. Update weights.
6. Repeat until error becomes minimum.

Efficient Python Code

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([[0,0],[0,1],[1,0],[1,1]])
Y = np.array([[0],[1],[1],[0]])

sig = lambda x: 1/(1+np.exp(-x))

wh = np.random.rand(2,2)
wo = np.random.rand(2,1)

lr = 0.1
errors = []

for i in range(10000):
    h = sig(X.dot(wh))
    o = sig(h.dot(wo))

    e = Y - o
    errors.append(np.mean(e**2))

    do = e * o * (1-o)
    dh = do.dot(wo.T) * h * (1-h)

    wo += h.T.dot(do) * lr
    wh += X.T.dot(dh) * lr

print(o)

plt.plot(errors)
plt.xlabel("Epochs")
plt.ylabel("Error")
plt.title("Back Propagation Learning")
plt.show()
```

Code Explanation

• NumPy is used for mathematical operations.
• Matplotlib is used for plotting the error graph.
• The XOR dataset is used for training.
• Sigmoid function introduces non-linearity.
• Random weights are initialized for hidden and output layers.
• Forward propagation computes output predictions.
• Error is calculated using actual and predicted outputs.
• Back propagation updates weights to minimize error.
• Error graph shows reduction in error during training.

Expected Output

The predicted output will be approximately:

[[0.01],
[0.98],
[0.98],
[0.02]]

The graph will show decreasing error as epochs increase.

How to Run the .py File in Anaconda

1. Open Anaconda Navigator.
2. Launch Spyder or Jupyter Notebook.
3. Save the program as backpropagation.py.
4. Install required libraries if needed:
pip install numpy matplotlib

5. Run the file using the Run button or press F5 in Spyder.
6. Observe predicted output and error graph.

How to Run the .ipynb File in Anaconda

1. Open Anaconda Navigator.
2. Launch Jupyter Notebook.
3. Open the .ipynb notebook file.
4. Run each cell one by one using Shift + Enter.
5. Observe the output and graph.

Advantages

• Learns complex non-linear patterns.
• Reduces prediction error automatically.
• Can be used in pattern recognition and classification problems.

Applications

• Image Recognition
• Speech Recognition
• Medical Diagnosis
• Stock Prediction
• Pattern Classification

Outcome

Successfully implemented a Feed Forward Neural Network using the Back Propagation algorithm and observed learning through error reduction.

Conclusion

The Feed Forward Neural Network successfully learned the XOR pattern using Back Propagation. The network continuously updated weights to reduce error and improve prediction accuracy.

Important Viva Questions with Answers

1. What is Back Propagation?
Back Propagation is a supervised learning algorithm used to train neural networks by reducing error.

2. What is a Feed Forward Neural Network?
It is a neural network where information flows from input to output without feedback connections.

3. Why is Sigmoid function used?
Because it produces output between 0 and 1 and helps introduce non-linearity.

4. What is XOR problem?
XOR is a non-linear logical problem commonly used to demonstrate neural network learning.

5. What is the role of hidden layer?
Hidden layers help solve complex non-linear problems.

6. What is an epoch?
One complete cycle of training through the dataset.

7. Why does error decrease during training?
Because weights are updated continuously using back propagation.

8. Which libraries are used?
NumPy and Matplotlib.

9. What is learning rate?
It controls the amount of weight update during training.

10. What does the graph represent?
It shows reduction in error during training.